

Project Proposal: Photo-to-Game-Asset Pipeline

1. Overview & Problem Statement

Right now, turning a real object into a usable game asset means photographing or scanning it, running it through a 3D generation tool by hand, fixing the rigging, texturing it, and then manually importing and setting it up inside a game engine. Every step is its own tool and its own learning curve, which is why small teams and solo developers rarely bother prototyping with real-world references.

We want to shrink that whole process down to: take a photo, pick an engine, get an asset that's ready to use. Single-photo 3D reconstruction, texturing, and auto-rigging already exist as services (Meshy, for example, does all three). What doesn't exist is the glue that connects them — something that figures out what kind of object it's looking at, sends the right request, and drops a properly set up asset (prefab, animator controller, collider) straight into the engine with nothing left to clean up by hand. That's the part we're building.

2. Data

This isn't a machine learning project in the usual sense — we're not training a model on a labeled dataset. What we do need is:

- **A test set of our own captures:** roughly 15-25 photos or short video scans of real objects, covering three kinds of things the pipeline has to handle differently — simple static props (a mug, a toy), rigid mechanical objects, and creature-like objects that need rigging. We'll shoot these ourselves with a phone under different lighting conditions so we're not just testing on easy, well-lit cases.
- **Reference labels for the classifier:** instead of training our own classifier, we're planning to use an existing pretrained vision model or a vision-capable LLM call to sort objects into categories (prop / creature / character). The actual work on our end is designing that prompt/label scheme and checking its accuracy against how we'd classify the same photos by hand.

On the pre-processing side: we'll reject obviously blurry or badly lit photos before they get used, and normalize image size/format before sending anything to the classifier or the Meshy API.

3. Plan of Action

3.1 Problem we're solving

Given one photo (or a short scan) of a physical object, output a game-ready 3D asset — mesh, textures, and a rig where it applies — that needs no manual fixing before it can be dropped into a scene.

3.2 How it works

1. **Capture** — a phone web app for taking and uploading the photo (React).
2. **Classification** — a pretrained vision model or LLM call sorts the object into prop / creature / character, which decides what settings we send to Meshy (pose type, rig style).
3. **Generation** — the backend calls the Meshy API (photo → mesh → texture → rig) and checks on the job until it's done.
4. **Desktop app** — a queue view with a 3D preview (three.js) and a button to pick which engine you're targeting.

5. **Import** — engine-specific scripts take the finished asset and set it up properly. For Unity, that means a C# Editor package that builds the prefab, Animator Controller, and collider automatically. Blender and Unreal are stretch goals.

3.3 Architecture

Phone web app (capture & upload) ↓ Backend (object classification + Meshy API orchestration) ↓ Desktop app hub (job queue, target picker, GUI) ↓ Target engine (Unity / Blender / Unreal)

3.4 Hardware capture track

A phone photo is the MVP capture method, but we're also working on richer capture as a side effort that plugs into the same pipeline: drone flyover scanning for objects that are too large, elevated, or awkward to capture by hand, and LiDAR/thermal/visual mapping for depth and heat data beyond a standard photo. Both feed the same classify-then-generate steps — a drone or sensor scan is just a richer input than a single photo. This could end up built directly into the app, or offered as its own separate mapping service sharing the same backend, depending on how it develops alongside the core MVP.

3.5 How we'll evaluate it

We're checking three things: - **Classification accuracy** — what percentage of our test photos get sorted correctly, checked against how we'd label them ourselves. - **End-to-end success rate** — out of our 15-25 test captures, how many come out the other end as a working Unity asset with nothing broken (rig, collider, prefab all correct)? We'll track why the ones that fail actually failed — bad classification, a bad Meshy result, or a bug in our import script. - **Time per asset** — how long it takes from uploading a photo to having a usable prefab, so we can actually show this saves time over doing it by hand.

We're not doing cross-validation or a train/test split here since there's no model being trained — we're testing whether the pipeline works reliably across different kinds of objects, not measuring prediction accuracy.

4. Timeline

Milestone	Target
Confirm the Meshy API actually works end-to-end (photo in, rigged FBX out), check pricing/rate limits, lock MVP scope to Unity	Week of Sep 29
Phone web app + backend upload/job endpoint	Early-mid October
Classifier wired in	Mid-October
Desktop app: job queue, target picker, preview	Late October
Unity Editor package: prefab/Animator Controller/collider automation	Early November
Full pipeline test across all captures; hardware integration spike	Mid-November
Stretch: Blender/Unreal import, if time allows	Late November
Final demo + writeup	Early December

5. What We Expect to Learn

- How well existing single-photo 3D reconstruction and auto-rigging tools actually hold up across object types they weren't specifically built for, and where they fall apart.
- Whether a simple classification step before generation actually improves results, versus just sending everything through with default settings.
- What it actually takes to build reliable engine-import automation (Unity Editor scripting, prefab/Animator Controller generation) that genuinely needs zero manual fixing.
- Where the pipeline breaks most often — bad photos, bad classification, or bad generation/import — so we know where the real problems are.

6. Team

Member	Owns
Cesar Salcido	Phone web app + backend API
Daniel Silvaz	Desktop app GUI + Unity Editor package